Subsystems

Operator Haptic Rendering Exoskeleton

Objective:
A wearable 1DOF finger exoskeleton that renders the robot's measured grip force back to the operator's finger as a controller resistance and vibration, while staying transparent during free-space motion. This lets the operator physically "feel" contact and grip force.

Interfaces:

- **Input:** Measured robot grip force via Communications; manual force input during standalone testing.
- **Output:** Measured finger position and subsystem status via Communications.
- **Operator:** Finger attachment transfers movement and resistive force.

Requirements:

- Measure finger opening consistently throughout the intended range of motion.
- Render stable resistance with low response delay and minimal unintended vibration.
- Allow easy free-space movement with high transparency.
- Provide adjustable finger attachment, travel limits, force/current limits, and emergency disable.
- Remove commanded resistance when communication is lost or a fault occurs.

Design Questions:

- Should the mechanism use a hinge or linear rail, and how will force reach the finger?
- Which actuator and transmission provide sufficient resistance while remaining easy to backdrive?
- Which position sensor should we use? **TBD: rotary or linear, depending on the mechanism.**
- What force range, travel range, response time, and control rate should we target?
- How should robot grip force map to finger resistance, and should vibration feedback be included?

Operator Hand Motion Tracking System

Objective:
Track the operator's hand pose and movement, including the end-of-arm (not including fingers) position and arm's orientation. This subsystem is directly responsible for vision-relying teleoperation of the arm.

Interfaces:

- Input:
  - Sensor data, which could be from an RGB camera, depth camera, and/or IMUs
- Output:
  - end-of-arm position coordinates (x, y, z) in units of length
  - rotation coordinates (roll, pitch, yaw) in degrees or radians
  - for phase 3: operator's full arm posture: shoulder angle, elbow angle, wrist angle, etc. (TBD; this requires a different tracking approach than endpoint/end-of-arm tracking alone)
- Interactions with other subsystems:
  - Communications System transfers the pose data to Robot 6DoF Arm Control System and data logs/tracking status to Operator UI + Telemetry
  - Robot 6DoF Arm Control System uses the pose data to move the robot arm to mirror the operator's hand
  - Operator UI + Telemetry displays data logs and tracking status
  - Operator Haptic Rendering Exoskeleton: no direct interactions but coupled through the operator. Finger tracking (opening and closing) will be handled by this subsystem.
  - Robot Force-Sensing End Effector: no interactions

Requirements:

- Synchronized data across all data streams
- Tracking jitter is low enough that arm motion mirroring doesn't feel jerky to the operator (e.g., sensor noise and minor operator tremor are smoothed)
- Sufficient rate of position and orientation tracking so the arm motion mirroring doesn't feel laggy.
- Position and orientation accuracy sufficient for the defined task precision.
- Calibration only needed at the start of the system and when ordered;
- Same operator hand pose should produce same tracked pose (repeatability)
- Defined tracking occlusion behaviour

Design Questions:

- Which motion tracking approach should we use?
  - Markered
    - Mocap cameras
    - LED camera tracking

        - Fiducial markers (ArUco / AprilTag, no IMU needed) with solvePnP algorithm
    - Markerless + IMU
        - Color/shape based object detection for position, IMU for orientation
        - Pose estimation (ML model, e.g., MediaPipe), with an optional IMU to fill in the gaps for lower latency and better accuracy if needed
- How many RGB-D cameras would we need? (TBD depending on motion tracking approach and taking into account occlusion)
- If we use an IMU:
    - What type of IMU should we use to correct drift in yaw?
        - 6-axis with comparison with the pose estimation from camera data
        - 9-axis
    - For the IMU, should we share the same MCU with the Exoskeleton?
        - If we're using a separate microcontroller, which one should we use:
            - wired to a host computer (e.g STM32, Teensy, Raspberry Pi Pico, Arduino Nano/Uno)
            - wireless/via WiFi (e.g ESP32, Raspberry Pi Pico W)
    - Which sensor fusion algorithm and filters should we use to reduce jitter without adding too much tracking delay?
        - Complementary filter
        - Kalman filter
- What are the required units for the position and orientation?
- What pose update rate does the Arm Control System's IK loop actually require?
- What position and orientation accuracy does the task require?
- Can the position coordinates in the operator's and robot's frame be 1:1, or do we need scaling? For example, scaling could be useful if the human's reachable workspace is smaller than the robot's
    - If we need scaling, who owns the scaling?
- What are the operator's expected behaviours? Can the operator reposition mid-session?
    - If so, should end-of-arm position be measured relative to the operator's current position (recalibrating/re-zeroing on reposition) or relative to their original position (allowing repositioning to extend reach, e.g. to reach farther objects)?
- How should the subsystem handle cases where the line of sight is blocked (occlusion)?
- For Phase 3:
    - Considering humans' arm rotation/movement limitations, is tracking the operator's full arm posture beneficial? (e.g in avoiding obstacles)
    - If we're using an IMU: How should the IMU (and therefore the MCU) be mounted to stay secure and consistently oriented across different operator wrist sizes?

Robot Force-Sensing End Effector

Objective: Design a gripper that can securely grasp and release objects while accurately measuring the gripping force applied to the object. The robot's physical interaction with objects to be fed back to the operator through haptic feedback.

Interfaces:
Mounted to the end of the 6DOF arm.
Input: Actuation commands for opening/closing the gripper.
Output: Gripper position/state and measured gripping force.

Communications System
Output: Measured grip force sent to the operator-side haptic exoskeleton.
Input: Gripper commands from the robot control system.

Haptic Exoskeleton
Measured gripping force is translated into resistive force and/or vibration at the operator's finger.

Objects being manipulated
Grippers must physically contact and hold objects without excessive deformation or slipping.

Requirements:
- Securely grips objects across the defined range of object sizes and shapes.
- Provides repeatable and measurable gripping force
- Force measurement has sufficient resolution and sensitivity to distinguish the object's stiffness
- Force measurements have low latency so haptic feedback does not feel delayed
- Gripper can open and close reliably with repeatable positioning
- Has sufficient gripping force and mechanical advantage for the maximum specified payload
- End effector mass is within the limit of the 6DOF arm's allowable torque
- Force sensor is located in an appropriate force path so measured force corresponds closely to actual object gripping force
- Wiring is protected against repeated arm and gripper motion

Design Questions:
- How many "fingers" should the gripper have?
- Which gripper finger corresponds to the operator's fingers to accurately relay the feeling of the object?
- What is the required range of object sizes?
- What is the maximum object mass/payload?
- What types of objects are we expecting to manipulate?
- What actuator should drive the gripper?
- Should the gripper automatically increase force if slipping is detected?

Robot 6DOF Arm Control System

Objective:

- Program a robust robot arm capable of receiving coordinate and orientation information from the hand motion tracking system and moving itself to mirror it.

Interfaces:

- Operator Haptic Rendering Exoskeleton: Medium through which operator will input commands to the robot arm
- Operator Hand Motion Tracking System: IMU and camera on the tracking system will tell the robot arm where to position and orient the end effector
- Robot Force-Sensing End Effector: End effector will be mounted at the end of the robot arm, with sufficient cable mounting and strain relief along the length of the arm. Mass and size of the end effector should be accounted for in IK calculations
- Communications System: Serves as ‘middleman’ between operator systems and arm control system
- Operator UI and Telemetry: Provides the operator additional feedback on where the robot arm is (live camera feed?)

Requirements:

- Safety features: Backdriveable motors, emergency stop included, system detects unexpected torque/current spikes and stops or reduces motion to prevent collisions
- Movement is low latency, low jitter
- Enables end effector to reach majority of the defined workspace in multiple orientations without operator intervention
- System has preventative measures against kinematic singularities
- Engineers have designed a repeatable procedure for homing all arm axes for calibration
- The same repeated operator glove movement should result in the same repeated robot end effector movement

Design Questions:

- Can mapping, IK, joint limits, collision avoidance, and control logic all be simulated without using the physical arm?
- Will joints use position, velocity, torque, or impedance control? Are the actuators capable of true torque control, or will torque have to be inferred from motor current? How much tuning will be required for different joints with different inertia and gearing?
- Can motion comfortably be 1:1 or does it need to be scaled?
- How will singularity points be handled?
- What is the maximum payload mass?

Communications System

Objectives:

- Provide a low-latency, low-jitter, bidirectional data link between the operator-side and robot-side subsystems
    - Allows position and force data to be exchanged in real time without breaking the stability of the haptic feedback loop

Interfaces:

- Operator Hand Tracking -> Robot Arm Control: hand/arm position + orientation, drives IK target pose
- Robot Force-Sensing End Effector -> Haptic Exoskeleton: measured grip force, drives actuator resistance
- Operator UI/Telemetry: read-only tap on both channels for dashboard/logging (Phase 3)

Requirements:

- Low latency (target TBD from Phase 0 testing)
- Low jitter — consistent delay matters more than raw speed
- Fail-safe on data loss (timeout) -> zero force, never stale force
- Force channel updates faster than position channel
- Ordered delivery -> discard out-of-order packets
- Locked interface contract (units, frame, schema) before other subteams build P1
- Live inspectable/loggable data, no custom tooling required

Design Questions:

- ROS2/DDS or raw UDP for the force channel (decide after measuring latency?)
- Wired, WiFi, or simulated remote link with injected delay?
- Units (mm/m, N/mN, degrees/radians?)
- Coordinate frame (operator, robot, or shared world frame?)
- Required force channel Hz (actuator-limited or human-perception-limited?)
- Required position channel Hz (IK/control-loop-limited?)
- How to measure one-way latency without synced clocks?

Operator UI + Telemetry

Objective:

Interfaces:

Requirements:

Design Questions: